HeadTracker — Pipeline de Visão Computacional

João Pedro Matos Pimenta · Abril 2026

O HeadTracker transforma ângulos da cabeça capturados por uma webcam em movimentos do cursor do rato. O sistema combina deteção de face, deteção de *landmarks* faciais, estimação de pose 3D e rastreamento ótico de pontos, tudo em tempo real a >30 fps numa CPU convencional.

```
Webcam/Vídeo → HeadTracker.update() (tracker.py) via rápida : LK optical flow + validação
forward-backward → solvePnP fallback : FaceDetector → MarkDetector → solvePnP (EPNP) →
OneEuroFilter (filters.py) → CursorController.update() (cursor.py) subtr. neutro → soft
deadzone → precision curve → pyautogui.moveTo
```

ETAPA 1 Deteção de Face — SCRFD

headtracker/tracking/face_detection.py

O modelo utilizado é o **SCRFD** (*Sample and Computation Redistributed Face Detection*), uma CNN ancorada exportada para ONNX e inferida via ONNX Runtime (GPU DirectML no Windows, CPU como fallback).

Arquitetura — Feature Pyramid Network (FPN)

A rede extrai mapas de características a três escalas diferentes (*strides* 8, 16 e 32 px). Cada nível da pirâmide é responsável por detetar faces a uma gama de tamanhos:

Stride	Área coberta	Faces detetadas
8 px	Alta resolução	Faces pequenas / próximas
16 px	Resolução média	Faces médias
32 px	Baixa resolução	Faces grandes / afastadas

Âncoras

Em cada célula da grelha de cada nível FPN existem 2 âncoras com dimensões pré-definidas. A rede prevê, para cada âncora, um *score* de confiança e 4 deltas (Δx, Δy, Δw, Δh) que ajustam a caixa delimitadora.

```
caixa_final = âncora + (Δx·w_âncora, Δy·h_âncora, Δw·..., Δh·...)
```

NMS — Non-Maximum Suppression

Após decodificar todas as caixas, filtra-se por confiança (> 0.7) e aplica-se NMS com *IoU threshold* 0.4: quando duas caixas se sobrepõem mais de 40 % em área, apenas a de maior confiança é mantida.

ETAPA 2 Deteção de Landmarks — CNN 68 pontos

headtracker/tracking/mark_detection.py

O recorte da face detetada é redimensionado para **128 × 128 px** e alimentado a uma CNN leve (ONNX). A saída da camada `dense_1` são 136 valores que representam as coordenadas normalizadas de 68 pontos faciais.

```
saída : [x₀, y₀, x₁, y₁, … , x₆₇, y₆₇]  ∈ [0, 1]

desnormalização:
  x_pixel = x_norm × largura_recorte + x_início_recorte
  y_pixel = y_norm × altura_recorte  + y_início_recorte
```

Distribuição dos 68 landmarks

Índices	Região
0 – 16	Contorno do queixo / mandíbula
17 – 26	Sobrancelhas
27 – 35	Nariz
36 – 47	Olhos
48 – 67	Boca / lábios

> **Decisão de design:** os índices 0–16 (queixo) e 48–67 (boca) são *excluídos* do solvePnP inicial (`STABLE_IDX = range(17, 48)`). Quando a boca abre, esses pontos deslocam-se para baixo; inclui-los faria o solver interpretar esse deslocamento como rotação de pitch, movendo o cursor indevidamente.

ETAPA 3 Modelo 3D da Cabeça e Câmara

headtracker/tracking/pose_estimation.py

Pontos 3D do modelo

O ficheiro `assets/model.txt` (derivado do OpenFace) contém as coordenadas 3D de referência dos 68 landmarks num espaço canónico (unidades em mm, origem no centro da face). O eixo Z é invertido (`Z × -1`) para alinhar com a convenção do OpenCV (Z aponta para a câmara).

Matriz intrínseca da câmara K

Sem calibração física da câmara, usa-se uma aproximação pela distância focal:

```
K = ⎡ f   0   c_x ⎤        f   = largura_imagem  (em píxeis)
    ⎢ 0   f   c_y ⎥        c_x = largura / 2
    ⎣ 0   0    1  ⎦        c_y = altura  / 2

coeficientes de distorção ≈ 0  (assumidos nulos)
```

A distância focal igual à largura é uma heurística que assume um campo de visão de ≈ 60°, razoável para a maioria das webcams.

ETAPA 4 Estimação de Pose — solvePnP

headtracker/tracking/tracker.py · pose_estimation.py

O **solvePnP** resolve o problema PnP (*Perspective-n-Point*): dado um conjunto de correspondências 2D↔3D e a matriz K, calcula a rotação e translação da câmara em relação ao objeto (ou equivalentemente, da cabeça em relação à câmara).

```
min  Σᵢ ‖ πK(R·Xᵢ + t) − xᵢ ‖²

onde:
  Xᵢ  = ponto 3D do modelo  (landmark i no espaço da cabeça)
  xᵢ  = ponto 2D observado  (landmark i na imagem)
  R   = matriz de rotação 3×3
  t   = vetor de translação 3×1
  πK  = projeção perspetiva com K
```

Dois modos de uso

Modo	Algoritmo	Quando é usado
Inicialização	EPNP	Primeiro frame ou após falha do rastreamento LK. Solução fechada, rápida e robusta sem estimativa inicial.
Rastreamento	ITERATIVE (Gauss-Newton)	A cada frame via LK. Parte da estimativa anterior como ponto inicial (`useExtrinsicGuess=True`), converge em 1-2 iterações.

ETAPA 5 Rodrigues e Ângulos de Euler

main.py · pose_estimation.py

O solvePnP devolve um **vetor de Rodrigues** `r_vec ∈ ℝ³`, que codifica eixo e ângulo de rotação num único vetor compacto.

```
θ = ‖r_vec‖       (ângulo de rotação, em radianos)
n = r_vec / θ    (eixo unitário de rotação)

Matriz de rotação (fórmula de Rodrigues):
R = I·cos(θ) + (1-cos(θ))·n·nᵀ + [n]ₓ·sin(θ)

onde [n]ₓ é a matriz anti-simétrica associada a n.
```

A seguir aplica-se `cv2.RQDecomp3x3(R)`, que faz a decomposição RQ de R e extrai os três ângulos de Euler:

```
angles = cv2.RQDecomp3x3(R)
pitch = angles[0]   (rotação em X — inclinação cima/baixo)
yaw   = angles[1]   (rotação em Y — virar esquerda/direita)
roll  = angles[2]   (rotação em Z — inclinar para o lado, ignorado)
```

> pitch e yaw são os únicos ângulos usados para controlo do cursor; roll é descartado.

ETAPA 6 Lucas-Kanade Optical Flow (via rápida)

headtracker/tracking/tracker.py — _track_lk()

Depois da inicialização, cada frame seguinte usa **optical flow piramidal de Lucas-Kanade** para rastrear os 68 pontos projetados sem re-executar a CNN. A pirâmide (4 níveis, janela 21×21) permite lidar com movimentos maiores.

Validação Forward-Backward

Correr LK só em frente (frame A → frame B) pode acumular erro. Usa-se a validação FB: rastreia-se também B → A e calcula-se o erro de ida e volta. Pontos com erro > 2.0 px são descartados.

```
Forward  : P_A  → LK →  P_B_fw       (A para B)
Backward : P_B_fw → LK →  P_A_bw       (B de volta para A)

Erro FB : ε_i = ‖P_A_i − P_A_bw_i‖

Ponto aceite se : ε_i < FB_THRESH = 2.0 px
                  status_fw == 1  (LK convergiu)
                  status_bw == 1
```

Critério de falha

Após o filtro FB, se menos de `MIN_TRACKED = 20` pontos sobreviverem, ou se o erro de reprojeção do solvePnP seguinte ultrapassar `REPROJ_THRESH = 8.0 px`, o rastreamento é declarado falhado e o sistema retorna à CNN (ETAPA 2).

```
reprojeção : proj_i = πK(R·X_i + t)
erro médio : e = mean(‖proj_i − obs_i‖)   para os inliers

se e ≥ 8.0 px → falha → reinicialização
```

ETAPA 7 Filtragem de Ângulos — One Euro Filter

headtracker/control/filters.py

Os ângulos brutos têm ruído de alta frequência. Um filtro passa-baixo fixo introduziria lag; o **One Euro Filter** adapta o corte dinamicamente: lento quando quieto (suaviza tremor), rápido quando o movimento é amplo (preserva reatividade).

```
dx      = (x - x_prev) / dt              (derivada estimada)
|dx|_f  = low_pass(|dx|, f_cutoff_der)   (derivada suavizada)

f_cutoff = min_cutoff + β × |dx|_f       (cutoff adaptativo)

x_filtered = low_pass(x, f_cutoff)

onde low_pass usa um filtro EWA de 1ª ordem:
  α = 2πf_cutoff·dt / (1 + 2πf_cutoff·dt)
  y = α·x + (1-α)·y_prev
```

Parâmetros usados: `min_cutoff = 0.5 Hz`, `β = 0.3`.

ETAPA 8 Mapeamento de Ângulos para o Cursor

headtracker/control/cursor.py

1. Subtração do neutro (calibração)

No primeiro frame detetado, os ângulos atuais são guardados como posição neutra. Em cada frame calcula-se o desvio em relação a esse neutro:

```
Δyaw   = yaw   − neutral_yaw
Δpitch = pitch − neutral_pitch
```

2. Soft deadzone

Uma zona morta suave absorve microtremores: para valores muito próximos de zero, a saída é atenuada quadraticamente (sem snap binário):

```
se |v| ≤ inner_dz  → saída = 0
se |v| ≥ outer_dz  → saída = v − sign(v)·inner_dz
senão               → saída = sign(v) · (|v| − inner_dz)² / (outer_dz − inner_dz)
```

3. Normalização

```
norm_x = clip(Δyaw   / max_yaw_deg,   -1, 1)    (max_yaw_deg   = 22°)
norm_y = clip(Δpitch / max_pitch_deg, -1, 1)    (max_pitch_deg = 13°)
```

4. Precision curve

Curva linear por partes: a região central (|norm| ≤ knee) tem ganho mais alto (navegação rápida); as bordas têm ganho mais baixo (precisão em alvos pequenos):

```
se |norm| ≤ knee:
    mapped = norm × (knee_out / knee)
senão:
    t = (|norm| − knee) / (1 − knee)
    mapped = sign(norm) × (knee_out + t × (1 − knee_out))
```

5. Conversão para coordenadas de ecrã

```
target_x = clip(W/2 + mapped_x × W/2,  margin, W-1-margin)
target_y = clip(H/2 − mapped_y × H/2,  margin, H-1-margin)
```

6. Suavização exponencial do movimento

Um EWA final elimina saltos bruscos entre frames. O alpha é proporcional à distância ao alvo — quanto mais longe, mais rápido se move:

```
dist  = ‖target − curr‖
alpha = clip(dist / 80.0,  0.30, 0.70)

curr_x += (target_x − curr_x) × alpha
curr_y += (target_y − curr_y) × alpha

se dist < 2.0 → snap direto (evitar vibração no centro)
```

EXTRA Reconhecimento de Voz — Vosk

headtracker/voice/engine.py · commands.py

Os comandos de voz correm em paralelo numa *thread* de fundo. O áudio é capturado pelo PyAudio (`CHUNK_SIZE = 800` amostras → ~50 ms de buffer) e colocado numa fila. A *thread* de reconhecimento retira amostras da fila e alimenta o `KaldiRecognizer` do Vosk.

O vocabulário é restrito às palavras que aparecem nos comandos definidos (`SetGrammar`), reduzindo drasticamente a taxa de erro. Os resultados parciais são processados *durante* a fala para minimizar latência. Um *cooldown* de 1.2 s impede execuções repetidas acidentais.

RESUMO

Diagrama do Pipeline

ENTRADA

Frame da webcam · *640 × 480 px, BGR, ~30–60 fps*

▼

ETAPA 1 — DETEÇÃO DE FACE

SCRFD ONNX · Feature Pyramid Network (strides 8 / 16 / 32)
âncoras → score de confiança (> 0.7) → NMS (IoU 0.4) → bounding box

▼

ETAPA 2 — LANDMARKS FACIAIS

CNN 68 pontos ONNX · recorte redimensionado para 128 × 128 px
dense_1 → 136 floats normalizados → 68 pontos 2D em coordenadas da imagem

▼

ETAPA 3 — MODELO 3D + CÂMARA

68 pontos 3D do modelo OpenFace (assets/model.txt, em mm)
Matriz K: f = largura, cc = (W/2, H/2) · distorção ≈ 0

▼

← FALLBACK (1.º frame ou falha LK) *(frames seguintes) VIA RÁPIDA →*

ETAPA 4A — solvePnP EPNP

31 landmarks estáveis (sobrancelhas + nariz + olhos)
solução fechada, sem estimativa inicial

ETAPA 6 → 4B — LK + solvePnP

Lucas-Kanade forward + backward (FB err < 2 px)
≥ 20 inliers → solvePnP ITERATIVE (reproj < 8 px)

▼

r_vec, t_vec

ETAPA 5 — RODRIGUES → ÂNGULOS DE EULER

cv2.Rodrigues: r_vec → R (3×3) · **cv2.RQDecomp3x3**: R → pitch, yaw, roll
pitch = inclinação vertical · yaw = rotação horizontal · roll ignorado

▼

ETAPA 7 — ONE EURO FILTER

Filtro passa-baixo adaptativo: cutoff sobe com a velocidade do movimento
min_cutoff = 0.5 Hz · β = 0.3 · elimina tremor sem introduzir lag

▼

ETAPA 8 — CURSOR CONTROLLER

Subtração do neutro → soft deadzone → normalização (±22° / ±13°)
Precision curve (centro rápido, bordas lentas) → suavização EWA (α 0.30–0.70)

▼

SAÍDA

pyautogui.moveTo(x, y) · *coordenada de ecrã em píxeis*

HeadTracker · João Pedro Matos Pimenta · 2026 · `github.com/Pimenta15/HeadTracker`